

Lesson 4

Hooks Continued, Events & Handlers

Code the Dream — React

Today's Session

EXPLORE

- `useEffect`: dependency array and cleanup
- `StrictMode` and double-invocation
- `useRef`: persisting values and DOM access
- Synthetic events and handler props
- Child-to-parent communication via callbacks

APPLY

- Fix five broken or incomplete components
- Topics: cleanup functions, `useRef` vs `useState`, event propagation, DOM focus, callback props

By the end: use `useEffect` safely, choose between `useRef` and `useState`, and wire up events so child components can communicate back to parents.

useEffect — Dependency Array

`useEffect` runs code **after** a component renders. The dependency array controls when it re-runs.

Dependency array	When the effect runs
Omitted	After every render
[] (empty)	Once — after the first render only
[dep1, dep2]	After first render, then whenever a dependency changes

```
useEffect(() => {  
  console.log('count changed:', count);  
}, [count]); // re-runs whenever count changes
```

useEffect — Cleanup Functions

Return a function from `useEffect` to clean up before the component unmounts or before the effect re-runs.

```
useEffect(() => {  
  const timer = setInterval(() => {  
    setCount(c => c + 1);  
  }, 1000);  
  
  return () => clearInterval(timer); // cleanup  
, []);
```

Without cleanup: the timer keeps running after the component is removed — a memory leak.

Always clean up: timers, event listeners, subscriptions, and open connections.

StrictMode & Double-Invocation

In development, StrictMode intentionally runs effects **twice**: mount → cleanup → mount.

This surfaces bugs in effects that aren't properly cleaned up.

Without cleanup

```
useEffect(() => {  
  setInterval(() => {  
    setCount(c => c + 1);  
  }, 1000);  
}, []);  
// StrictMode leaves 2 intervals  
// running → count jumps by 2
```

With cleanup

```
useEffect(() => {  
  const t = setInterval(() => {  
    setCount(c => c + 1);  
  }, 1000);  
  return () => clearInterval(t);  
}, []);  
// cleanup clears the first interval  
// → count increments correctly
```

useRef — What It Does

useRef stores a value that **persists across renders** without triggering a re-render when it changes.

```
const ref = useRef(initialValue);  
// read:  ref.current  
// write: ref.current = newValue  (no re-render)
```

When to use useRef instead of useState:

	useState	useRef
Triggers re-render	Yes	No
Use for	Values displayed in UI	Values not shown in UI
Common uses	Counters, lists, form data	Timers, DOM nodes, previous values

useRef — DOM Access

The most common use of `useRef` is to get a direct reference to a DOM element.

```
import { useRef } from 'react';

function SearchBar() {
  const inputRef = useRef();

  function focusInput() {
    inputRef.current.focus();
  }

  return (
    <>
      <input ref={inputRef} type="text" />
      <button onClick={focusInput}>Focus</button>
    </>
  );
}
```

Attach `ref` to an element. React sets `ref.current` to the DOM node after render.

useRef — Rules

- **Read and write** `ref.current` **inside handlers or effects only** — not directly in the render body
- **Don't render ref values in JSX** — if a value needs to appear in the UI, use `useState` instead

```
// ❌ Don't do this – reads ref during render
function Bad() {
  const count = useRef(0);
  return <p>{count.current}</p>; // won't update when count changes
}
```

```
// ✅ Do this – useState drives the display
function Good() {
  const [count, setCount] = useState(0);
  return <p>{count}</p>;
}
```


Check for Understanding

A component tracks how many times an API response has been received. This count is **only used in a `console.log`** inside a `useEffect` — it never appears in the UI. Which hook is the better choice?

- A) `useState` — it's the standard way to store values in React
- B) `useRef` — the count persists across renders but doesn't need to trigger a re-render
- C) A plain `let` variable — hooks are unnecessary here
- D) `useEffect` — you can store the count in the dependency array

Synthetic Events

React wraps native browser events in a **synthetic event** object — a normalized wrapper that works consistently across all browsers.

Commonly used properties and methods:

<code>event.target</code>	The element where the event was triggered
<code>event.currentTarget</code>	The element where the listener is attached
<code>event.target.value</code>	The current value of an input field
<code>event.preventDefault()</code>	Stops the browser's default behavior (e.g. page refresh on form submit)
<code>event.stopPropagation()</code>	Stops the event from bubbling up to parent elements

Handler Props & Event Bubbling

React's built-in handler props listen for events on an element. All are prefixed with `on`.

```
<button onClick={handleClick}>Click me</button>
<input onChange={handleChange} />
<form onSubmit={handleSubmit}>...</form>
```

Event bubbling: when an event fires on an inner element, it travels up through all parent elements unless stopped.

```
<div onClick={handleOuterClick}>          {/* fires second */}
  <button onClick={handleInnerClick}>      {/* fires first */}
    Click
  </button>
</div>
```

Call `event.stopPropagation()` inside `handleInnerClick` to prevent `handleOuterClick` from firing.

Child-to-Parent Communication

Props flow **down**. To communicate back **up**, pass a function as a prop — the child calls it when something happens.

```
// Parent – owns state, defines the handler, passes it down
function Parent() {
  const [count, setCount] = useState(0);
  function increment() { setCount(c => c + 1); }
  return <Child onIncrement={increment} />;
}
```

```
// Child – receives the handler, calls it on click
function Child({ onIncrement }) {
  return <button onClick={onIncrement}>+1</button>;
}
```

Convention: name callback props starting with `on` — e.g. `onAddTodo`, `onIncrement`, `onClose`.

Handler Composition Options

React gives you flexibility in how you write handler functions.

```
// Named function (defined separately – clearest for complex logic)
function handleClick() { setCount(c => c + 1); }
<button onClick={handleClick}>+</button>
```

```
// Inline arrow function (fine for simple one-liners)
<button onClick={() => setCount(c => c + 1)}>+</button>
```

```
// Passing arguments (use inline arrow to wrap the call)
<button onClick={() => handleAddItem(item.id)}>Add</button>
```

Prefix custom handler names with **handle** — e.g. `handleClick`, `handleSubmit`, `handleAddTodo`. It signals that this function responds to an event.

Check for Understanding

A parent component has this state and handler:

```
const [items, setItems] = useState([]);  
function addItem(name) { setItems(prev => [...prev, name]); }
```

It renders a child: `<AddButton onAdd={addItem} />`. What should the child look like?

A) `function AddButton() { return <button onClick={addItem('hat')}>Add</button>; }`

B) `function AddButton({ onAdd }) { return <button onClick={() => onAdd('hat')}>Add</button>; }`

C) `function AddButton({ addItem }) { return <button onClick={addItem}>Add</button>; }`

D) `function AddButton() { addItem('hat'); return <button>Add</button>; }`

Now: Apply

Five components. Five concepts. Let's fix them.

This Week's Exercise — Five Fixes

Open `src/exercises/lesson-04/` and work through each file.

File	Concept
<code>BugStrictMode.jsx</code>	<code>useEffect</code> cleanup (timer)
<code>FindCorrectHook.jsx</code>	<code>useRef</code> VS <code>useState</code>
<code>BugEventPropagation.jsx</code>	<code>stopPropagation</code>
<code>FillRefFocus.jsx</code>	DOM access with <code>useRef</code>
<code>BugChildParentCommuncation/</code>	Callback prop communication

For each: read the comment at the top, identify the problem, fix it, and write a short explanation at the bottom of the file.

Exercise 1 — StrictMode Timer Bug

The interval is missing a cleanup function. StrictMode mounts twice, leaving two intervals running — the count jumps by 2.

Buggy

```
useEffect(() => {  
  setInterval(() => {  
    setCount(c => c + 1);  
  }, 1000);  
}, []);
```

Fixed

```
useEffect(() => {  
  const t = setInterval(() => {  
    setCount(c => c + 1);  
  }, 1000);  
  return () => clearInterval(t);  
}, []);
```

The cleanup runs when StrictMode unmounts the first instance, clearing that interval before the second mount starts.

Exercise 2 — useRef VS useState

A plain `let` variable resets to `0` on every render — clicks never accumulate. The fix depends on the goal.

Buggy

```
let clickCount = 0;

function handleClick() {
  clickCount++;
}
// clickCount resets each render
// button text never updates
```

Fixed

```
const [clickCount, setClickCount] =
  useState(0);

function handleClick() {
  setClickCount(c => c + 1);
}
// state persists + triggers re-render
// button text updates correctly
```

Since `clickCount` is **displayed** in the button, it must use `useState`. `useRef` would persist the value but the display would never update.

Exercise 3 — Event Propagation

Clicking the button fires `handleInnerClick`, then bubbles up and fires `handleOuterClick` too.

Buggy

```
function handleInnerClick() {  
  alert('Button Clicked ✅');  
}
```

Fixed

```
function handleInnerClick(event) {  
  event.stopPropagation();  
  alert('Button Clicked ✅');  
}
```

`stopPropagation()` stops the event from traveling up to the outer div's `onClick` handler.

Exercise 4 — Focus an Input with useRef

`focusInput` is empty — clicking the button does nothing. Wire up a ref to imperatively focus the input.

Buggy

```
function focusInput() {}

return (
  <div>
    <input type="text" />
    <button onClick={focusInput}>
      Focus Input
    </button>
  </div>
);
```

Fixed

```
const inputRef = useRef();

function focusInput() {
  inputRef.current.focus();
}

return (
  <div>
    <input ref={inputRef} type="text" />
    <button onClick={focusInput}>
      Focus Input
    </button>
  </div>
);
```

Exercise 5 — Child-to-Parent Communication

Child's button has an empty handler and no connection to Parent's state.

Buggy

```
// Parent.jsx
<Child />

// Child.jsx
export default function Child() {
  return (
    <button onClick={() => {}}>
      Increment Counter
    </button>
  );
}
```

Fixed

```
// Parent.jsx
<Child onIncrement={increment} />

// Child.jsx
export default function Child({ onIncrement }) {
  return (
    <button onClick={onIncrement}>
      Increment Counter
    </button>
  );
}
```

Wrap-Up

This week you learned:

- `useEffect` cleanup prevents memory leaks — always clean up timers, listeners, and connections
- `StrictMode`'s double-invocation exposes missing cleanup in development
- `useRef` persists values without re-rendering — use it for DOM access and behind-the-scenes values
- Handler props wire events to functions; callbacks let children communicate back to parents
- `event.stopPropagation()` prevents events from bubbling up to parent elements

Before next session: Complete the weekly assignment — make your todo form functional using `onSubmit`, `useRef` for input focus, and a handler that calls `addTodo`. Submit your PR link in CTD Learns. Post questions in **`#[class-name]-discussion`** on Slack.